WIRELESS COMMUNICATION NETWORKS AND SYSTEMS

LAB 03

Digital Modulation (ASK, 2-FSK, BPSK)

Learning Objectives

- Explain how bits are mapped to waveforms using ASK, 2-FSK, and BPSK.
- Generate modulated signals, visualize them in time and frequency.
- Build simple receivers (energy/envelope/correlation) to recover bits.
- Measure Bit Error Rate (BER) vs SNR on an AWGN channel and compare schemes.

Background (Very Short)

Digital modulation maps bits onto a *carrier* by changing its amplitude (ASK), frequency (2-FSK), or phase (BPSK). In this lab we keep math light and emphasize intuition, plots, and code. We use an AWGN channel to study robustness.

Global Parameters & Notation

- Bit rate R_b (bits/s), bit duration $T_b = 1/R_b$.
- Oversampling factor N_s (samples per bit), sampling freq. $f_s = N_s \cdot R_b$.
- Carrier frequency f_c (ASK/BPSK). For 2-FSK, use f_0, f_1.

Recommended defaults (safe for plots):

$$R_b = 100 \text{ b/s}, \quad N_s = 100, \quad f_s = 10\,\text{kHz}, \quad f_c = 1\,\text{kHz}, \quad (f_0, f_1) = (800, 1200)\,\text{Hz}$$

Step 0 — Setup & Helper Functions

```python
import numpy as np
import matplotlib.pyplot as plt

# ---- Global parameters (feel free to tweak) ----
Rb  = 100       # bit rate [bits/s]
Ns  = 100       # samples per bit
fs  = Rb*Ns     # sampling frequency [Hz]
Tb  = 1/Rb      # bit duration [s]
fc  = 1000      # carrier for ASK/BPSK [Hz]
f0, f1 = 800, 1200   # FSK mark/space [Hz]

def nrz_bits(bits):
    """Map bits {0,1} to NRZ levels {0,1} for ASK (OOK)."""
```

Copyright © WCNS Team 2026/2027

```
    return np.array(bits, dtype=float)

def nrz_bipolar(bits):
    """Map bits {0,1} to {-1,+1} for BPSK."""
    return 2*np.array(bits, dtype=float) - 1.0

def upsample(symbols, Ns):
    """Repeat each symbol Ns times (NRZ rectangular pulse shaping)."""
    return np.repeat(symbols, Ns)

def time_axis(n_samples):
    return np.arange(n_samples)/fs

def carrier_wave(freq, n_samples, phase=0.0):
    t = time_axis(n_samples)
    return np.sin(2*np.pi*freq*t + phase)

def rfft_mag(x):
    X = np.fft.rfft(x)
    f = np.fft.rfftfreq(len(x), d=1/fs)
    return f, np.abs(X)/len(x)

def add_awgn(x, snr_db):
    """Add AWGN to reach target SNR (per-sample power basis)."""
    p_sig = np.mean(x**2)
    snr_lin = 10**(snr_db/10)
    p_noise = p_sig/snr_lin
    noise = np.random.normal(0, np.sqrt(p_noise), size=len(x))
    return x + noise

def ber(ref_bits, rx_bits):
    ref = np.array(ref_bits).astype(int)
    rx  = np.array(rx_bits).astype(int)
    return np.mean(ref != rx)
```

Step 1 — Bitstream, Timebase & Sanity Plots

Task. Generate $N_b = 200$ random bits, build NRZ waveforms (rectangular pulses) and visualize them.

```
Nb = 200
bits = np.random.randint(0, 2, size=Nb)

# NRZ for ASK (0/1) and BPSK (-1/+1)
nrz_ask  = nrz_bits(bits)
nrz_bpsk = nrz_bipolar(bits)

# Upsample to waveform-level sequences
base_ask  = upsample(nrz_ask,  Ns)      # length = Nb*Ns
base_bpsk = upsample(nrz_bpsk, Ns)

t = time_axis(len(base_ask))

# Plot a small window (first 10 bits) for readability
win = 10*Ns
plt.figure(); plt.plot(t[:win], base_ask[:win])
plt.title("NRZ baseband (ASK OOK levels)"); plt.xlabel("Time [s]"); plt.ylabel("Amp")

plt.figure(); plt.plot(t[:win], base_bpsk[:win])
```

Copyright © WCNS Team 2026/2027

```
plt.title("NRZ baseband (BPSK levels -1/+1)"); plt.xlabel("Time [s]"); plt.ylabel("Amp")
plt.show()
```

Record: Screenshots of the two baseband waveforms (first ∼10 bits) with short captions.

Step 2 — ASK (OOK) Modulation & Envelope Detection

Modulator. For On-Off Keying, multiply NRZ levels $\{0, 1\}$ by a carrier at f_c.

```
c = carrier_wave(fc, len(base_ask))
s_ask = base_ask * c  # OOK

# Time view (short window)
plt.figure(); plt.plot(t[:win], s_ask[:win])
plt.title("ASK (OOK) waveform"); plt.xlabel("Time [s]"); plt.ylabel("Amplitude")
plt.show()

# Spectrum (use a moderate slice for speed or whole sequence)
f, M = rfft_mag(s_ask)
plt.figure(); plt.plot(f, M); plt.title("ASK spectrum (magnitude)")
plt.xlabel("Frequency [Hz]"); plt.ylabel("|X(f)|"); plt.show()
```

Channel + Receiver (envelope detection). Rectify & integrate per bit, threshold at θ.

```
snr_db = 6
y_ask = add_awgn(s_ask, snr_db)

# Simple envelope: absolute value then integrate energy per bit
env = np.abs(y_ask)
rx_bits_ask = []
for i in range(Nb):
    seg = env[i*Ns:(i+1)*Ns]
    metric = np.mean(seg)  # average envelope in bit window
    rx_bits_ask.append(1 if metric > 0.3 else 0)  # threshold tweakable

print("ASK (OOK) BER @", snr_db, "dB  =", ber(bits, rx_bits_ask))
```

Expected outputs: ASK time plot, spectrum, chosen threshold, BER at $0, 4, 8, 12$ dB.

Step 3 — 2-FSK Modulation & Noncoherent Energy Detection

Modulator. Use f_1 for bit 1, f_0 for bit 0 (same amplitude).

```
# Build per-bit FSK by stitching bit-length carriers
s_fsk = np.zeros(Nb*Ns)
for i, b in enumerate(bits):
    fbit = f1 if b==1 else f0
    s_fsk[i*Ns:(i+1)*Ns] = carrier_wave(fbit, Ns)

plt.figure(); plt.plot(t[:win], s_fsk[:win])
plt.title("2-FSK waveform"); plt.xlabel("Time [s]"); plt.ylabel("Amplitude"); plt.show()

f, M = rfft_mag(s_fsk)
plt.figure(); plt.plot(f, M); plt.title("2-FSK spectrum"); plt.xlabel("Frequency [Hz]")
```

Copyright © WCNS Team 2026/2027

```
plt.ylabel("|X(f)|"); plt.show()
```

Channel + Receiver (energy in narrowband filters). Compare energy around f_0 vs f_1.

```
def tone_energy(x, f_tone):
    # Correlate with in-phase and quadrature sinusoids, sum energies in a bit window
    tloc = np.arange(Ns)/fs
    cI = np.sin(2*np.pi*f_tone*tloc)
    cQ = np.cos(2*np.pi*f_tone*tloc)
    E = 0.0
    for i in range(Nb):
        seg = x[i*Ns:(i+1)*Ns]
        I = np.dot(seg, cI)
        Q = np.dot(seg, cQ)
        E += (I**2 + Q**2)  # optional: keep per-bit
    return E

snr_db = 6
y_fsk = add_awgn(s_fsk, snr_db)

rx_bits_fsk = []
tloc = np.arange(Ns)/fs
cI0, cQ0 = np.sin(2*np.pi*f0*tloc), np.cos(2*np.pi*f0*tloc)
cI1, cQ1 = np.sin(2*np.pi*f1*tloc), np.cos(2*np.pi*f1*tloc)

for i in range(Nb):
    seg = y_fsk[i*Ns:(i+1)*Ns]
    I0, Q0 = np.dot(seg, cI0), np.dot(seg, cQ0)
    I1, Q1 = np.dot(seg, cI1), np.dot(seg, cQ1)
    E0, E1 = I0**2 + Q0**2, I1**2 + Q1**2
    rx_bits_fsk.append(1 if E1 > E0 else 0)

print("2-FSK BER @", snr_db, "dB =", ber(bits, rx_bits_fsk))
```

Expected outputs: FSK time plot, spectrum, BER at $0, 4, 8, 12$ dB.

Step 4 — BPSK Modulation & Coherent Correlator Receiver

Modulator. Map bits to $\{-1, +1\}$, then multiply by carrier at f_c.

```
s_bpsk = upsample(nrz_bipolar(bits), Ns) * carrier_wave(fc, Nb*Ns)

plt.figure(); plt.plot(t[:win], s_bpsk[:win])
plt.title("BPSK waveform"); plt.xlabel("Time [s]"); plt.ylabel("Amplitude"); plt.
    show()

f, M = rfft_mag(s_bpsk)
plt.figure(); plt.plot(f, M); plt.title("BPSK spectrum"); plt.xlabel("Frequency [Hz]
    ")
plt.ylabel("|X(f)|"); plt.show()
```

Channel + Receiver (coherent correlation). Correlate per bit with *known* carrier.

```
snr_db = 6
y_bpsk = add_awgn(s_bpsk, snr_db)

rx_bits_bpsk = []
cbit = carrier_wave(fc, Ns)   # reference carrier for one-bit window
```

Copyright © WCNS Team 2026/2027

```

for i in range(Nb):
    seg = y_bpsk[i*Ns:(i+1)*Ns]
    metric = np.dot(seg, cbit)        # integrate over bit
    rx_bits_bpsk.append(1 if metric >= 0 else 0)

print("BPSK BER @", snr_db, "dB =", ber(bits, rx_bits_bpsk))
```

Expected outputs: BPSK time plot, spectrum, BER at $0, 4, 8, 12$ dB.

Step 5 — BER vs SNR (Mini Experiment)

Task. Sweep SNR and compare schemes *on the same bit sequence*.

```
SNRs = [0, 2, 4, 6, 8, 10, 12]
ber_ask, ber_fsk, ber_bpsk = [], [], []

# Precompute fixed waveforms (before noise)
s_ask  = upsample(nrz_bits(bits), Ns)    * carrier_wave(fc, Nb*Ns)
s_fsk  = np.concatenate([carrier_wave(f1 if b else f0, Ns) for b in bits])
s_bpsk = upsample(nrz_bipolar(bits), Ns) * carrier_wave(fc, Nb*Ns)
cbit   = carrier_wave(fc, Ns)
tloc   = np.arange(Ns)/fs
cI0, cQ0 = np.sin(2*np.pi*f0*tloc), np.cos(2*np.pi*f0*tloc)
cI1, cQ1 = np.sin(2*np.pi*f1*tloc), np.cos(2*np.pi*f1*tloc)

for snr_db in SNRs:
    # ASK receive
    y = add_awgn(s_ask, snr_db); env = np.abs(y)
    rxA = []
    for i in range(Nb):
        m = np.mean(env[i*Ns:(i+1)*Ns])
        rxA.append(1 if m > 0.3 else 0)
    ber_ask.append(ber(bits, rxA))

    # FSK receive
    y = add_awgn(s_fsk, snr_db); rxF = []
    for i in range(Nb):
        seg = y[i*Ns:(i+1)*Ns]
        I0,Q0 = np.dot(seg,cI0), np.dot(seg,cQ0)
        I1,Q1 = np.dot(seg,cI1), np.dot(seg,cQ1)
        rxF.append(1 if (I1**2+Q1**2) > (I0**2+Q0**2) else 0)
    ber_fsk.append(ber(bits, rxF))

    # BPSK receive
    y = add_awgn(s_bpsk, snr_db); rxP = []
    for i in range(Nb):
        seg = y[i*Ns:(i+1)*Ns]
        rxP.append(1 if np.dot(seg, cbit) >= 0 else 0)
    ber_bpsk.append(ber(bits, rxP))

print("SNRs:", SNRs)
print("BER ASK :",  ber_ask)
print("BER 2FSK:",  ber_fsk)
print("BER BPSK:",  ber_bpsk)

# Optional: plot BER curves (linear y for simplicity)
plt.figure(); plt.plot(SNRs, ber_ask, marker='o', label='ASK (OOK)')
plt.plot(SNRs, ber_fsk, marker='o', label='2-FSK')
```

Copyright © WCNS Team 2026/2027

```
plt.plot(SNRs, ber_bpsk, marker='o', label='BPSK')
plt.xlabel("SNR [dB]"); plt.ylabel("BER"); plt.title("BER vs SNR (AWGN)")
plt.legend(); plt.grid(True); plt.show()
```

Record: The printed BER lists and the BER plot. **Explain** which scheme is most robust and why.

Copyright © WCNS Team 2026/2027

Step 6 — Optional Explorations (Pick One)

- **Constellation view (BPSK).** Scatter of correlator outputs (I) across noise levels.
- **Pulse shaping.** Replace rectangular NRZ with raised-cosine (conceptual) and discuss bandwidth.
- **Carrier offset.** Add a small frequency offset to BPSK carrier at the receiver and observe BER.

Guiding Questions

1. Why does BPSK typically outperform ASK at the same SNR?
2. When might 2-FSK be preferred over BPSK (hint: noncoherent detection, narrowband interference)?
3. How do carrier frequency choices (f_c, f_0, f_1) and bit rate (R_b) affect spectral occupancy?
4. What practical issues would appear in a real radio (timing, phase/frequency offsets, filtering)?

Copyright © WCNS Team 2026/2027